

JavaScript Revision

this keyword

```
const user = {  
  name: "Alex Carry",  
  greet: function(){  
    console.log(this.name);  
  }  
}
```

```
user.greet(); // Alex Carry
```

```
//we are just assigning a reference to the function  
const greetLater = user.greet;
```

```
// to solve this problem  
const greetLater = user.greet.bind(user);  
greetLater() // undefined
```

```
/* output:  
  Alex Carry  
  undefined  
*/
```

Why the first one prints "Alex Carry" and second one "undefined"

- **For `user.greet()`** : When you call a function *as a method* of an object (using the dot notation `user.greet()`), JavaScript sets the `this` context for that specific call to be the object to the left of the dot. In this case, `this` becomes `user` , so `this.name` is "Alex".

- **For `greetLater()`** : When you do `const greetLater = user.greet;` , you're just assigning a *reference* to the function to the `greetLater` variable. All the information about it being part of the `user` object is lost. When you call `greetLater()` directly, it's just a plain function call. In this situation (in non-strict mode), `this` defaults to the global object (like `window` in a browser), which doesn't have a `.name` property, leading to `undefined` .
- the `bind()` method of `Function` instances creates a new function that, when called, has its `this` keyword set to a provided value. so bind does not run the function. It just creates a new function that is pre-set with `this` value you want.
- `call()` / `apply()` both immediately execute the function right now with this value provide.
- `call()` this method calls the function immediately. The first argument you pass to `.call()` is the `this` context.
- `apply()` accepts arguments as a single array

```
const user = { name: "Alex" };
```

```
function say(greeting, punctuation) {
  console.log(greeting + ", " + this.name + punctuation);
}
```

```
// Using .call(): Arguments are passed one-by-one
say.call(user, "Hello", "!"); // Logs "Hello, Alex!"
```

```
// Using .apply(): Arguments are passed in an array
say.apply(user, ["Hi", "."]); // Logs "Hi, Alex."
```

Method	Executes Immediately?	How Arguments are Passed	Returns
<code>.bind()</code>	No	(Can be pre-filled)	A new function
<code>.call()</code>	Yes	Individually (e.g., <code>arg1</code> , <code>arg2</code>)	The function's result

<code>.apply()</code>	Yes	As an Array (e.g., <code>[arg1, arg2]</code>)	The function's result
-----------------------	-----	---	-----------------------

Promises and async/await

- Promises are like telling that the thing is going to happen but you have to wait for it.

Real life analogy

1. You get a receipt (the Promise) immediately.
2. The receipt isn't the food, but it's a guarantee you will get something.
3. Your order is "pending" while the kitchen works.
4. Eventually, the promise is "fulfilled" (you get your food) or "rejected" (they're out of ingredients).

- `async` tells JavaScript that the function might have time consuming operations in it.
- `await` keyword pauses the execution of only that async function right at the line. in simple words it means "pause this function and let me know when the promise is done"
 - while one function is paused, JavaScript is not blocked. It can leave the current function and go do other things. run other code, respond to user clicks

Array methods

- `map()` is for transforming each item into something new
- `filter()` is for selecting a subset of items
- `reduce()` is used to get a single value out of an array

Challenge 1

1. You have to fetch a list of users from a fake API and then find all the users who are over 30 years old.

Setup:

1. an API function: this function returns a promise that resolves with a list of users. we have to `await` it
2. GOAL: write an async function that:
 - a. fetches the users
 - b. uses an array method to filter the list down to only users older than 30
 - c. logs the resulting array of names

Basic code snippet to start with

```
/*  
1. An API function: this function returns a promise that resolves with a list of u  
sers. we have to `await` it  
  
GOAL: write an async function that:  
1. fetches the users  
2. uses an array method to filter the list down to only users older than 30  
3. logs the resulting array of names  
*/  
  
function fetchUsers() {  
  console.log("fetching users...");
```

```

return new Promise((resolve) => {
  setTimeout(() => {
    resolve([
      { name: "Alex", age: 28 },
      { name: "Maria", age: 42 },
      { name: "Sam", age: 19 },
      { name: "Ben", age: 35 },
    ]);
  }, 1000); // Simulates 1-second network delay
});
}

// 1. How do you get the users from fetchUsers()?
// 2. How do you filter the users array?
// 3. How do you log the result?
async function findUsers() {
  // stop here and get the all the users
  const allUsers = await fetchUsers();

  const filteredUsers = await allUsers
    // filter to get the item you want.
    .filter((user) => user.age > 30)
    // map to transform those items into required shape you want.
    .map((user) => user.name);
  console.log(filteredUsers)
}

findUsers();

```

ES6 modules - import/export

- You `export` code from one file and `import` it into another.

There are two types of exports

1. Named exports (multiple possible per file)
2. Default exports (only can have one per file)

ES6 Classes

- Class is like a blueprint for creating objects

it basically bundles two things:

1. Data
 2. Function/methods
- before es6, we used "constructor functions" to do this, but class keyword is just cleaner, "syntactic sugar"

It basically contains two things

- constructor: this is a special method that gets called automatically whenever you create a new object from the class.
 - it is used to set up the object's initial properties.
- methods: functions that live inside the class and can work with the object's data.

```
class User{
  constructor(name, age){
    // this refer to the new object being created
    this.name = name;
    this.age = age;
  }
}
```

```

    greet(){
        console.log(`My name is ${this.name} and am ${this.age} years old`)
    }
}

// this is how we can create an object
const first = new User("Ben", 20);
const second = new User("John", 21);

// this is how we can objects and their methods
second.greet();
first.greet();

```

Inheritance

```

class User{
    constructor(name, age){
        this.name = name
        this.age = age
    }

    greet(){
        console.log(`My name is ${this.name} and am ${this.age} years old`)
    }
}

class Admin extends User{
    editWebsite(){
        console.log(`${this.name} is editing the website`)
    }
}

const newAdmin = new Admin("Ashish", 21)

```

```
newAdmin.greet()  
newAdmin.editWebsite()
```

Usage of super keyword

```
class User{  
  constructor(name){  
    this.name = name  
  }  
}  
  
class Admin extends User{  
  constructor(name, permissionLevel){  
    super(name)  
    this.permissionLevel = permissionLevel  
  }  
}  
  
const admin = new Admin("Ashish", "full-access")  
console.log(admin.name)  
console.log(admin.permissionLevel)
```

- `super(name)`: this line calls the User class's constructor and passes the name argument to it.
- We must use `super()` before we can use the `this` keyword in a child class constructor

Challenge 2

- Our goal is to fetch a list of products, find only the ones in the "Electronics" category, and then apply a 10% increase to them

- GOAL: Write this async function
1. Fetch the products
 2. Filter them to get ONLY "Electronics"
 3. Map the filtered array to a new array where each product's price has a 10% increase. (price * 1.1)
 4. Log the final array of new, updated electronics.

```
function fetchProducts() {
  console.log("Fetching products...");
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve([
        { id: 1, name: "Laptop", category: "Electronics", price: 1200 },
        { id: 2, name: "Coffee Mug", category: "Kitchen", price: 25 },
        { id: 3, name: "Smartphone", category: "Electronics", price: 800 },
        { id: 4, name: "Bookshelf", category: "Furniture", price: 150 },
        { id: 5, name: "Headphones", category: "Electronics", price: 200 },
      ]);
    }, 1000);
  });
}

async function getUpdatedElectronics() {
  const products = await fetchProducts();
  const updatedProducts = products
    .filter((product) => product.category === "Electronics")
    .map((product) => {
      return {
        ...product,
        price: product.price * 1.1,
      };
    });
}
```

```
console.log(updatedProducts);  
}  
getUpdatedElectronics();
```